

ІПЗ

Server-side Application

ЛАБОРАТОРНА РОБОТА IV

■ Мета роботи

Реалізація серверного додатку за вказаними вимогами задля забезпечення логування повідомлень, формування черги повідомлень, яка опрацьовується окремими потоками серверного додатку. Дослідження та реалізація моделей організації взаємодії Long Polling та Short Polling.

■ Теми та техніки що використовуються

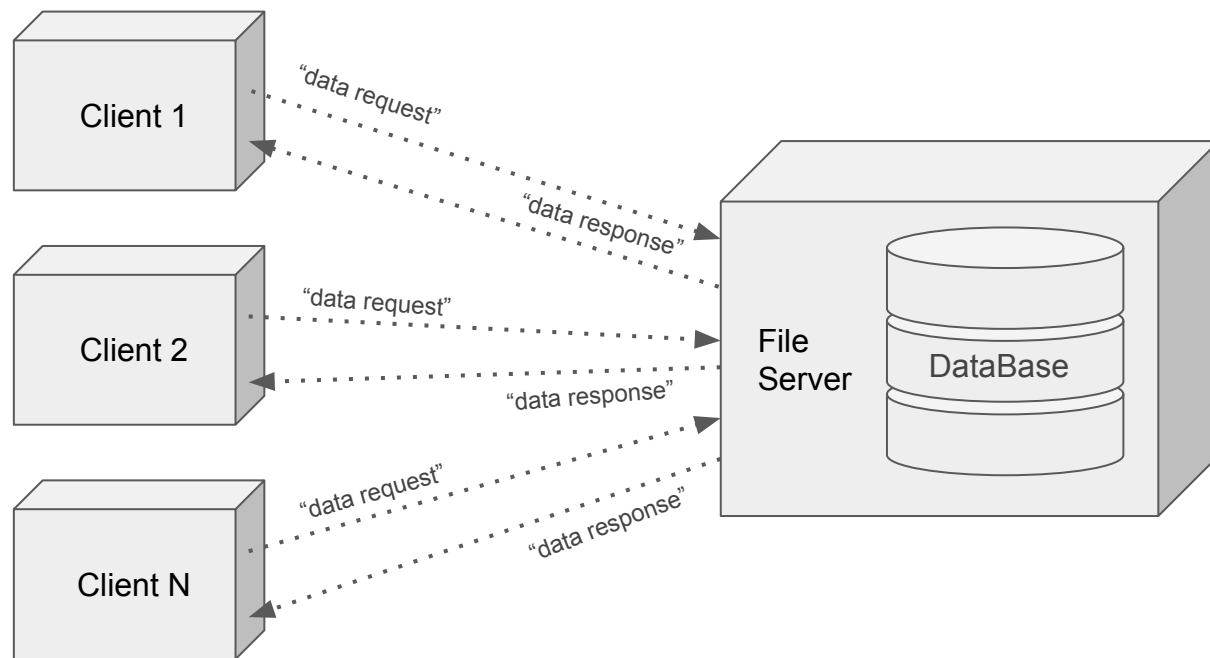
- види клієнт-серверних архітектур
- розробка багатопоточного серверного додатку
- реалізація взаємодії за принципами “тонкий клієнт” або “товстий клієнт”
- реалізація Short Polling або Long Polling

- **Клієнт-серверна архітектура** - найбільш розповсюджений патерн, який використовується при створенні складних програмних засобів, що передбачає підтримку великої кількості споживачів. Головна ідея патерну полягає у розділенні функціональності програмного засобу на дві частини: клієнтську та серверну, а також використання мережі для організації зв'язку між ними

- **Поширені види клієнт-серверної архітектури**
 - Однорівнева клієнт-серверна архітектура
 - Дворівнева клієнт-серверна архітектура
 - Трирівнева клієнт-серверна архітектура
 - N-рівнева клієнт-серверна архітектура

■ Однорівнева клієнт-серверна архітектура

- Передбачає відсутність центрального серверу, який регламентує роботу клієнтів, обробляє їх запити та надає клієнтам дані
- Всі клієнти мають однакові функціональні можливості на рівні права
- Всі клієнти звертаються до одного файлового серверу або одного серверу бази даних
- Сервер БД дуже вразливий до великої кількості запитів та, здебільшого, не займається валідацією клієнтських запитів
- Часто виникають проблеми синхронізації даних через велику кількість варіантів даних на різних клієнтах



■ Однорівнева клієнт-серверна архітектура

■ Переваги

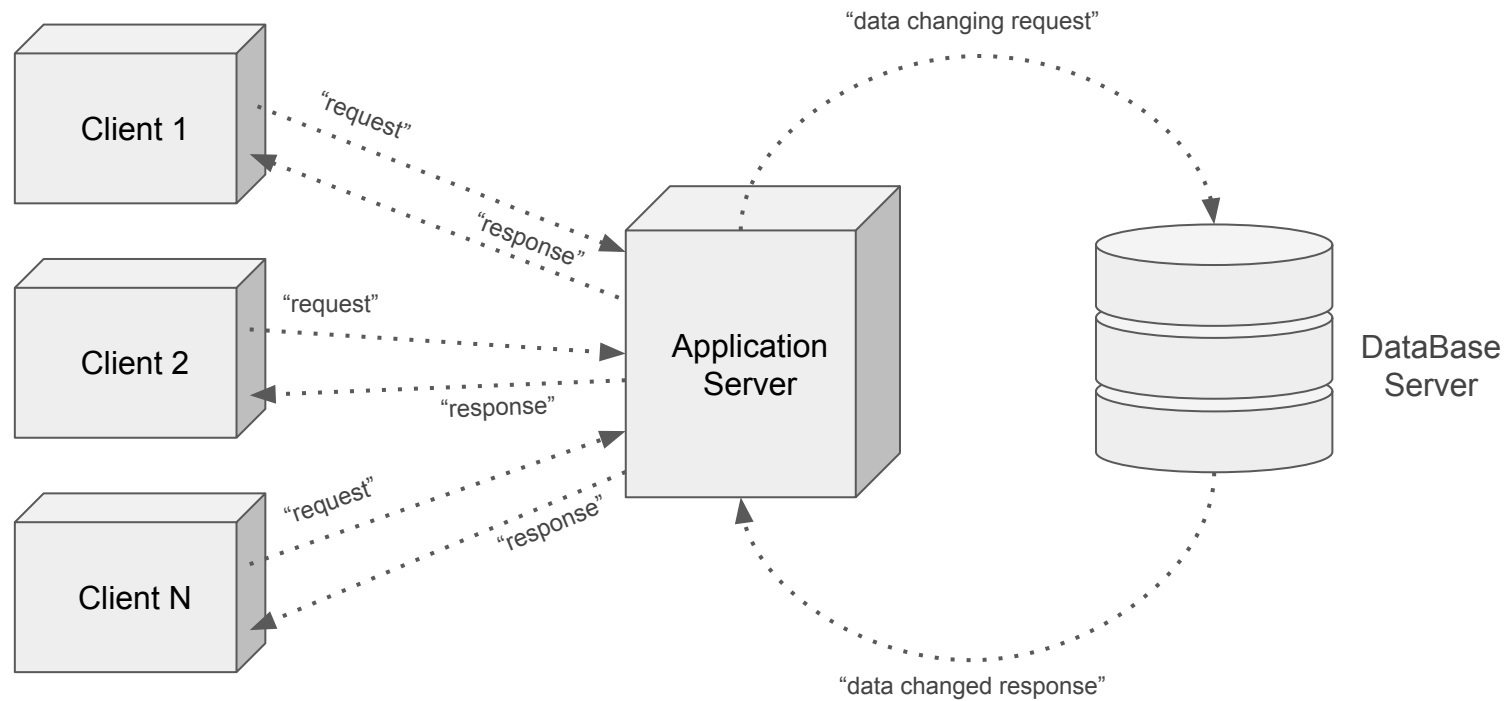
- простота реалізації та розгортання
- відносно невеликі вимоги до файл-серверної частини
- немає механізму розподілення ролей для доступу до інформації

■ Недоліки

- проблеми з синхронізацією даних між великою кількістю клієнтів
- висока вразливість системи при діях зловмисників
- низькі можливості масштабування кількості клієнтських додатків

■ Дворівнева клієнт-серверна архітектура

- Передбачає існування серверного додатку / Application Server, який може проводити валідацію запитів від клієнтів
- Найбільш поширена архітектура для роботи з невеликим за функціональністю програмними засобами
- Передбачає можливості роботи у одному з двох варіантів:
 - “fat client thin server”, коли клієнтський додаток має багато повноважень
 - “thin client fat server”, коли клієнтський додаток майже не має повноважень
- Система дуже вразлива до збоїв на частині Application Server



■ Дворівнева клієнт-серверна архітектура

■ Переваги

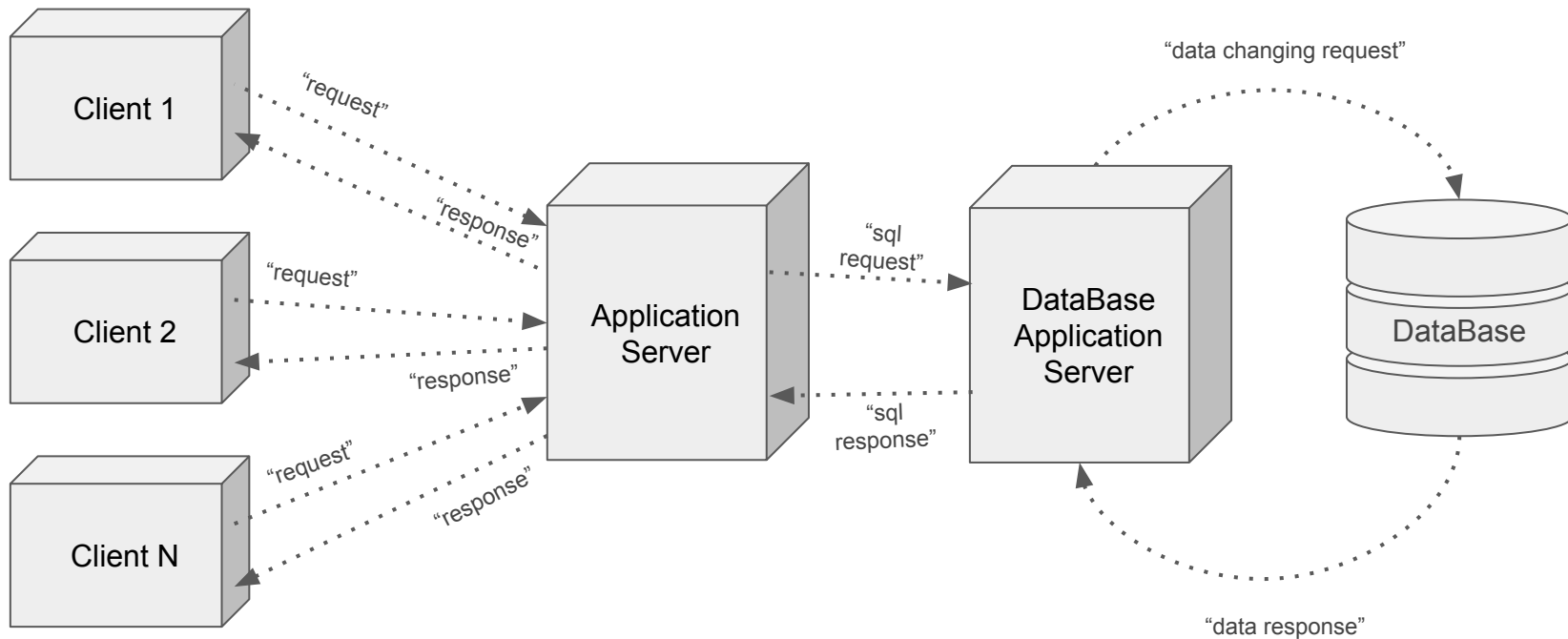
- простота реалізації та розгортання
- відносно низька вартість
- оптимальна для організації невеликих за функціональністю систем
- має більший захист ніж однорівнева клієнт-серверна архітектура

■ Недоліки

- проблеми масштабування клієнтських додатків
- висока крихкість - у разі виходу серверу з ладу - не працює вся система
- низька безпечність у режимі “fat client thin server”

■ Трирівнева клієнт-серверна архітектура

- Передбачається розділення серверної функціональності на частину ApplicationServer та частину DataBaseApplicationServer
- Частина ApplicationServer безпосередньо співпрацює з клієнтськими додатками та не має прямого доступу до бази даних
- Частина ApplicationServer виконує первинну валідацію запитів клієнтських додатків на виконання тих чи інших функцій
- Частина DataBaseApplicationServer безпосередньо співпрацює базою даних, формує запити на зміну даних, версіонує клієнтів та передає оновлені стани клієнтів на частину ApplicationServer



■ Трирівнева клієнт-серверна архітектура

■ Переваги

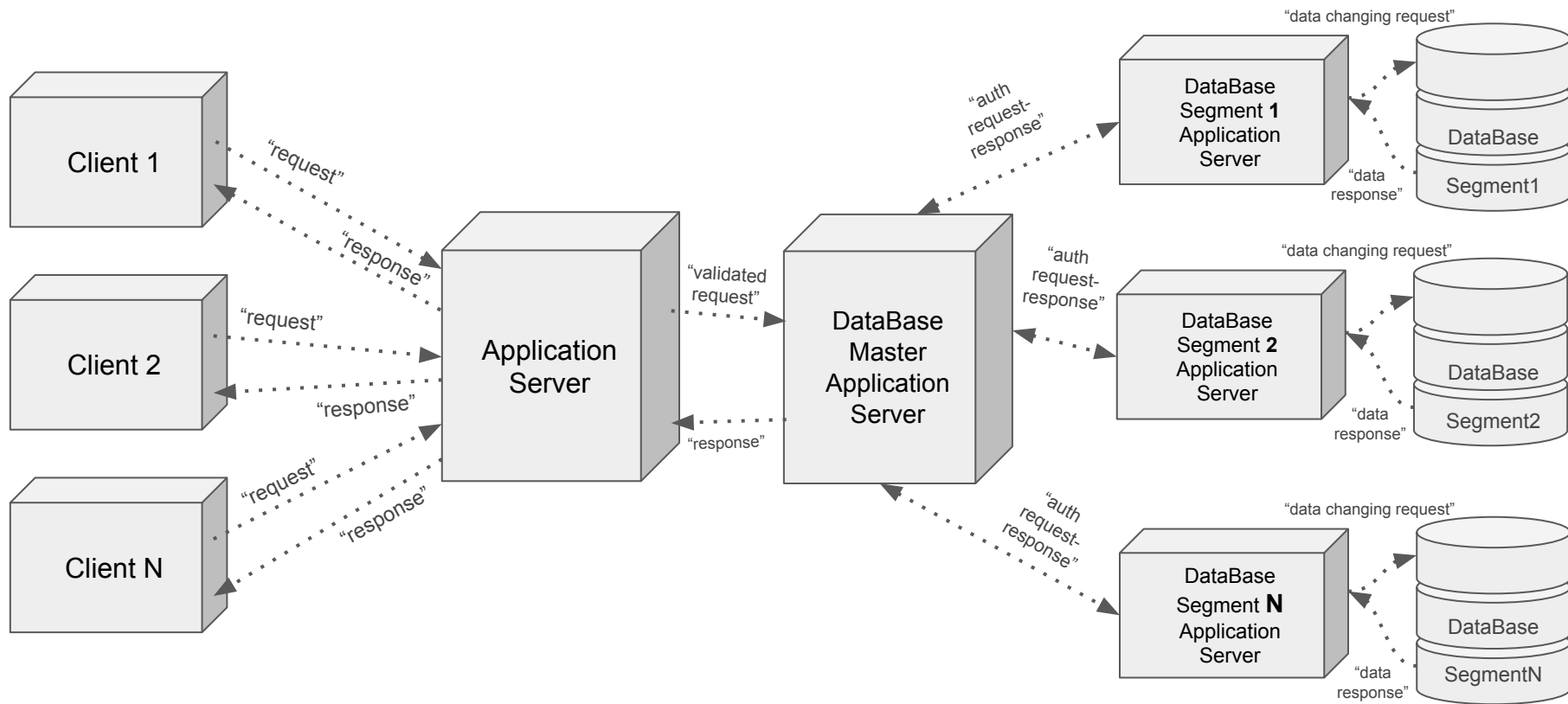
- забезпечення гарного захисту даних від підміни або дублювання sql-запитів
- забезпечення захисту від клонування клієнтів
- простота адміністрування
- гарна синхронізація даних

■ Недоліки

- збільшення вартості реалізації порівняно із дворівневими архітектурами
- підвищення технічних вимог до кваліфікації співробітників
- вразливість системи до проблем на рівнях серверних частин додатків

■ N-рівнева клієнт-серверна архітектура

- Передбачається розділення серверної функціональності на частину ApplicationServer та частину DataBaseMasterApplicationServer, а також приховані рівні сегментів баз даних
- Частина ApplicationServer безпосередньо співпрацює з клієнтськими додатками та виконує первинну валідацію запитів клієнтських додатків на виконання тих чи інших функцій
- Частина DataBaseMasterApplicationServer співпрацює з додатками сегментів, які співпрацюють з окремою частиною розподіленої бази даних
- Частини DataBaseSegmentApplicationServer формує запити на зміну даних, версіонує клієнтів та передає оновлені стани клієнтів на частину маєстру



■ N-рівнева клієнт-серверна архітектура

■ Переваги

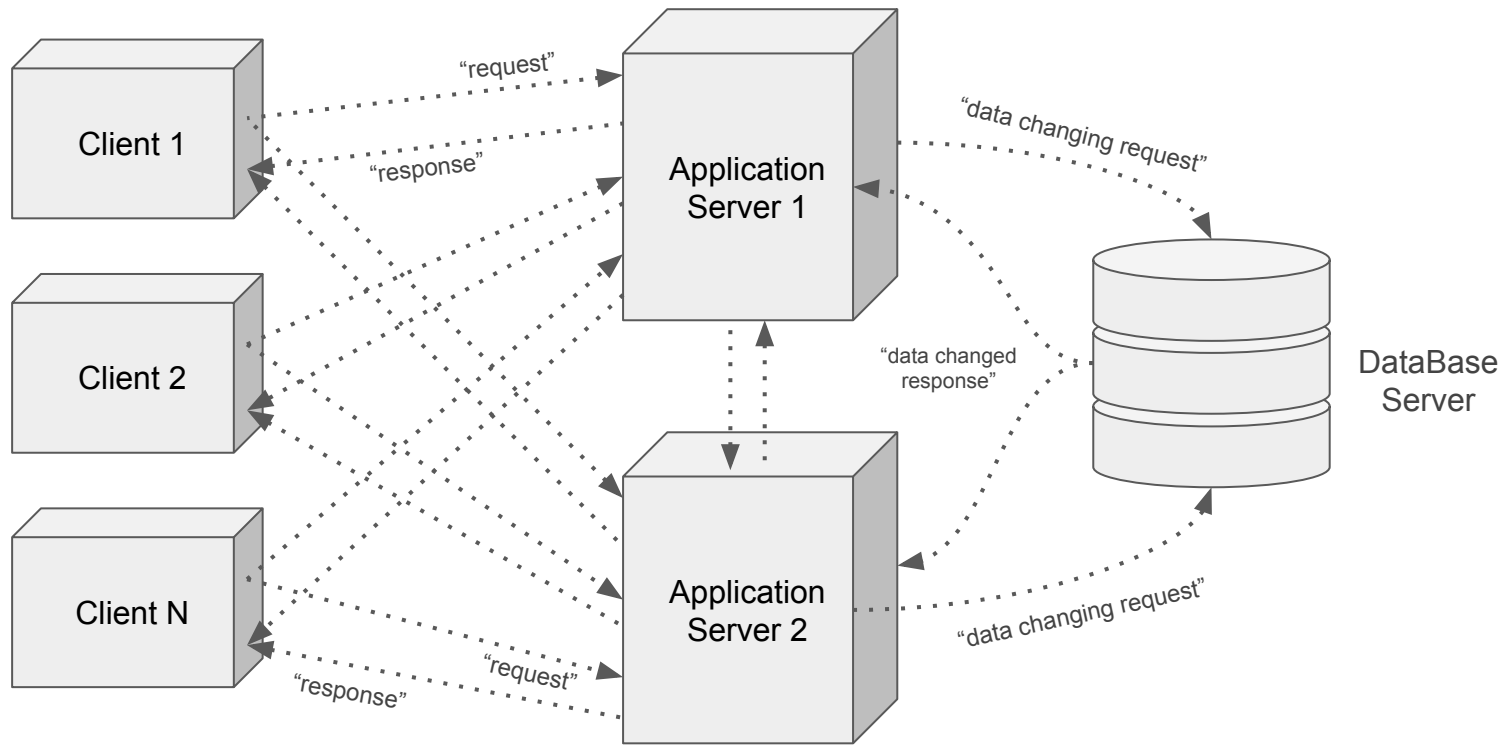
- розподілене зберігання даних
- підвищення захисту системи від зловмисницьких дій користувачів
- підвищення швидкості обробки запитів користувачів
- покращення можливостей масштабування

■ Недоліки

- висока складність системи
- висока вартість системи
- високі вимоги до кваліфікації співробітників

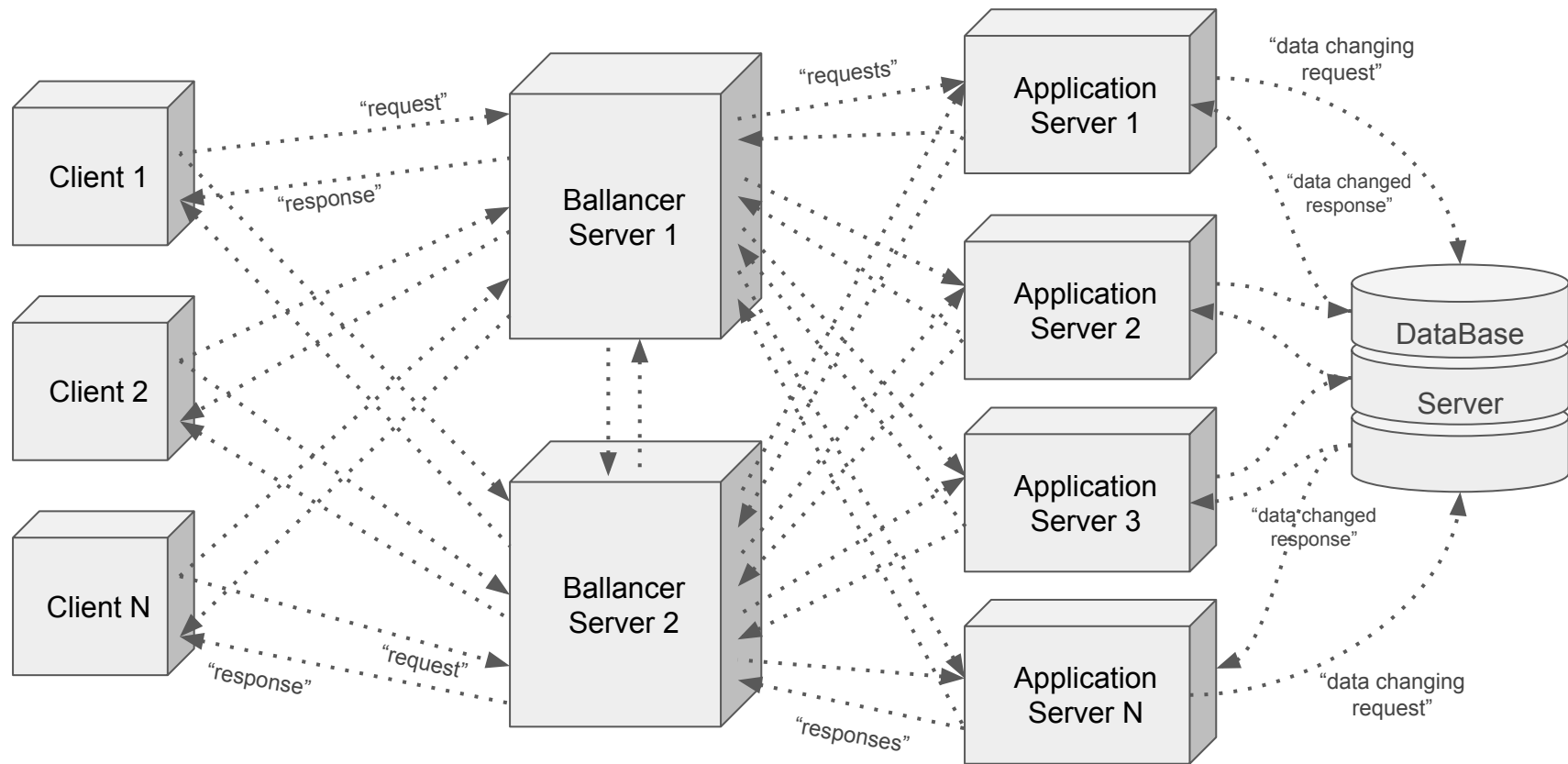
■ Кластери серврів

- На другому рівні системи виділяється декілька кластерів, які займаються обробкою запитів з клієнтів
- Сервери, що формують кластер, перерозподіляють між собою об'єми клієнтських запитів та можуть замінювати один одного у випадку виникнення проблем на одному з кластерів
- Використовуються для підвищення надійності клієнт-серверної системи
- Будь-які сервери кластеру можуть обмінюватися повідомленнями між собою та з базою даних
- Будь-які сервери кластеру можуть обмінюватися повідомленнями з клієнтами



■ Балансувальники навантаження на сервери

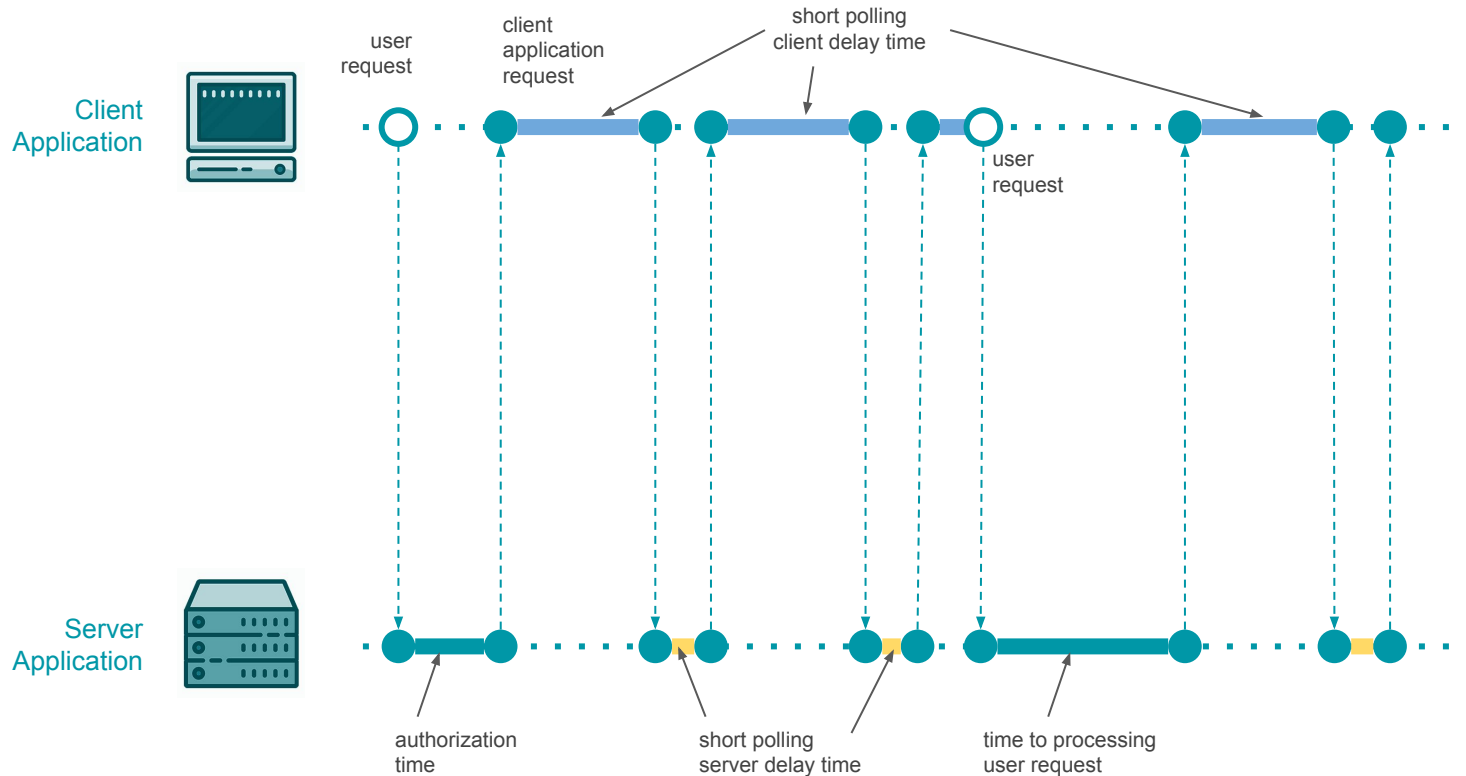
- Балансувальники використовуються для регулювання навантаження у вигляді клієнтських запитів та їх обробки між серверами системи
- Балансувальник приймає запити від клієнтів та перенаправляє їх на вільний сервер для подальшої обробки
- Балансувальних не валідує запити від клієнтів
- Балансувальних не співпрацює з базою даних
- Задля підвищення безпечності роботи система зазвичай використовує декілька балансувальників, які пов'язані між собою та всіма серверами, що займаються опрацюванням клієнтських запитів



- **Transmission Control Protocol / Internet Protocol** - набір протоколів, що використовуються для передавання даних, який отримав назву від двох належних до нього протоколів - TCP та IP
 - **HTTP** - протокол передавання гіпертексту
 - **HTTPS** - розширення HTTP для підтримки шифрування з метою підвищення безпеки завдяки криптографічним протоколам SSL та TLS
 - **SSL / Secure Sockets Layer** - безпечний криптографічний протокол
 - **FTP / File Transfer Protocol** - протокол передавання файлів з файл-серверу
 - **POP3 / Post Office Protocol** - протокол передачі поштових повідомлень
 - **SMTP / Simple Mail Transfer Protocol** - протокол передачі пошти
 - **DTN** - протокол для мереж космічного зв'язку IPN, яким користується NASA

- **HTTP / HTTPs** - протокол, який дозволяє отримувати ресурси з мережі, наприклад HTML-документи. Цей протокол є основою обміну даними у глобальній мережі, та використовується при організації взаємодії між частинами клієнт-серверних програмних засобів
 - Запити до сервера ініціюються клієнтом / одерживачем (web-browser)
 - Документи передаються частинами, клієнт уточнює та запитує у сервера необхідні частини документів для роботи
 - Кожне передавання даних має підтвердження
 - Використовує специфічні порти - 80 для HTTP, та 443 для HTTPs

- **Polling** - спосіб організації взаємодії між клієнтським та серверним додатком, який передбачає формування клієнтом регулярних запитів на отримання або зміну даних сервером.
- **Short Polling** - техніка, відповідно до якої клієнт надсилає повідомлення на сервер із запитом на доступ або зміну даних через фіксовані затримки часу після отримання відповіді на попередньо надісланий запит; сервер при цьому миттєво відповідає на запити клієнтів
 - Клієнт надсилає запит на сервер
 - Сервер відповідає порожньою відповіддю або даними
 - Клієнт очікує деякий визначений час та знову відправляє запит до сервера
 - Сервер відповідає порожньою відповіддю або даними



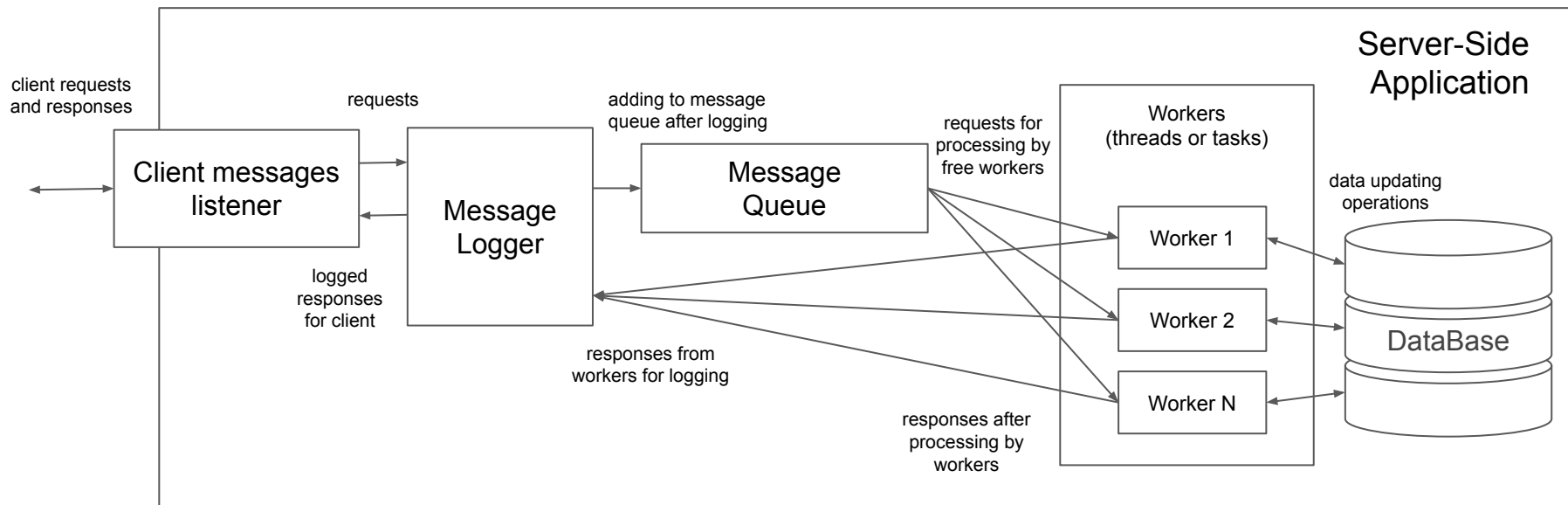
- **Long Polling** - техніка, відповідно до якої клієнт надсилає повідомлення на сервер із запитом на доступ або зміну даних через фіксовані затримки часу після отримання відповіді на попередньо надісланий запит, але очікує відповіді від сервера протягом невизначеного часу
 - Клієнт надсилає запит на сервер
 - Сервер відповідає коли в нього з'являються дані, або допоки не спливе термін очікування
 - Клієнт надсилає запит на сервер
 - Сервер відповідає коли в нього з'являються дані, або допоки не спливе термін очікування
 - Сервер відповідає коли в нього з'являються дані, або допоки не спливе термін очікування

Технічне завдання

- Реалізувати принцип клієнт-серверної взаємодії “тонкий клієнт” для проекту, обраного на ЛБ 1, та опрацьованого впродовж наступних лабораторних робіт
- Реалізувати обмін даними між клієнтським та серверним додатком за допомогою tokenів замість пересилання інформації авторизованого користувача на кожному запиті: у випадку успішної авторизації сервер формує унікальний token з літер латинського алфавіту та цифр, який від передає як ідентифікатор цього клієнта у подальших повідомленнях. Таким чином, клієнт у наступних повідомленнях серверну має використовувати цей унікальний token замість того, щоб постійно передавати свій email та пароль у кожних повідомленнях
- Реалізувати ідею Short Polling або Long Polling на клієнтській та серверній частинах додатку

Технічне завдання / загальна схема серверної частини

- Реалізувати серверну частину додатку за загальною схемою, наведеною нижче



Технічне завдання

- Реалізувати логування клієнтських повідомлень та серверних відповідей на серверній частині додатку
- Провести рефакторинг коду клієнтської та серверної частин додатку відповідно до обраного завдання
- Сформулювати UML-діаграми серверної частини програмного засобу на рівні компонентів та на рівні класів

Звіт повинен мати

- мова звіту має бути українською
- титульна сторінка з назвою лабораторної роботи та інформацією про команду виконавців
- мета лабораторної роботи
- код виконаного технічного завдання з поясненнями
- UML-діаграма на рівні компонентів для обраної методології Short Polling або Long Polling
- UML-діаграма серверного додатку
- висновки

Кількість балів за ЛБ: 1 - 8

■ Контрольні запитання

- В чому різниця між дворівневою та трирівневою клієнт-серверними архітектурами?
- В чому особливості функціонування N-рівневих клієнт-серверних архітектур?
- У яких випадках краще застосовувати дворівневу клієнт-серверну архітектуру?
- Наведіть недоліки N-рівневої клієнт-серверної архітектури
- Наведіть переваги дворівневої клієнт-серверної архітектури
- Що розуміють під поняттям сервлет та у яких мережах він використовується?
- Поясніть особливості використання кластеру серверів

■ Контрольні запитання

- Поясніть особливості використання балансувальників серверів
- Порівняйте переваги та недоліки використання кластерів серверів та балансувальників серверів
- Що розуміють під сегментацією бази даних?
- У чому відмінності між HTTPs та UDP протоколами?
- Що таке токен та навіщо це використовується у клієнт-серверній взаємодії?
- У чому відмінність між Peer-to-Peer Pattern та Master-Slave Pattern?
- Чому патерни MVP, MVC та MVVM не використовуються при розробці серверних частин програмних засобів?

Дякую за увагу!